TASK DOCKER-02

Deployed a Sample Application on Tomcat Using Docker

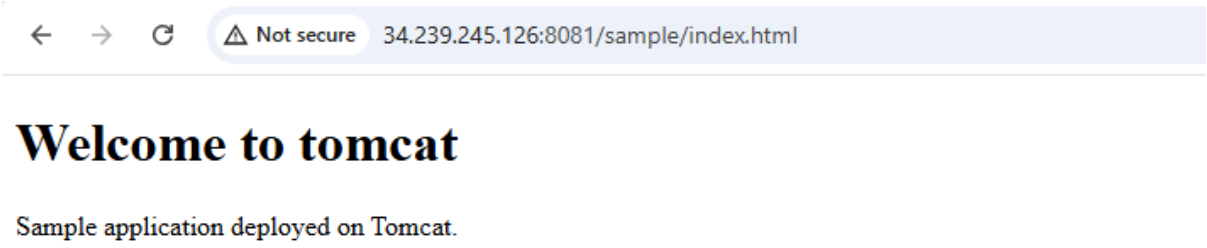
Objective: To run a Tomcat container and deploy a sample web application that can be accessed through a browser.

Steps performed:

1. Created a Tomcat container using the `tomcat:9.0` Docker image.
2. Mapped host port `8081` to container port `8080`.
3. Created a sample HTML application displaying:
   - Welcome to Tomcat
   - Sample application deployed on Tomcat.
4. Packaged the application into `sample.war`.
5. Copied `sample.war` into the Tomcat container's `webapps` directory.
6. Verified that Tomcat deployed the application successfully.

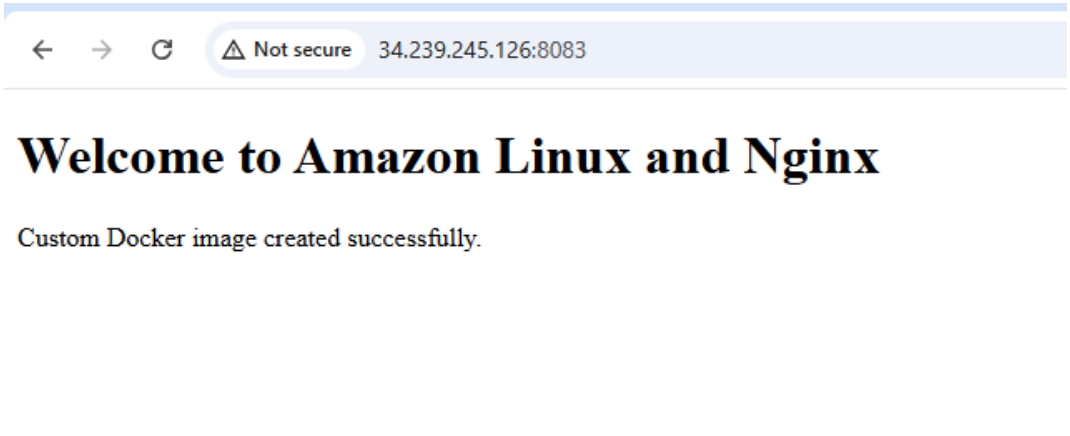
```
Successfully copied 2.56kB to tomcat-8081:/usr/local/tomcat/webapps/
ubuntu@ip-172-31-27-223:~/tomcat/app$ sudo docker exec tomcat-8081 ls -lh /usr/local/tomcat/webapps/
total 8.0K
drwxr-x--- 3 root   root   4.0K Oct 13 11:28 sample
-rw-rw-r-- 1 ubuntu ubuntu  550 Oct 13 11:27 sample.war
ubuntu@ip-172-31-27-223:~/tomcat/app$ curl -i http://localhost:8081/sample/
HTTP/1.1 200
Accept-Ranges: bytes
ETag: W/"181-1791889082000"
Last-Modified: Tue, 13 Oct 2026 10:58:02 GMT
Content-Type: text/html
Content-Length: 181
Date: Tue, 13 Oct 2026 11:30:39 GMT

<!DOCTYPE html>
<html>
<head>
    <title>tomcat  Application</title>
</head>
<body>
    <h1>Welcome to tomcat</h1>
    <p>Sample application deployed on Tomcat.</p>
</body>
</html>
ubuntu@ip-172-31-27-223:~/tomcat/app$
```



Created a Custom Docker Image Using Amazon Linux and Nginx

Objective: To build a custom Docker image using Amazon Linux, install Nginx, and serve a custom HTML page.




```
ubuntu@ip-172-31-27-223:~$ sudo docker run -d \
  --name amazonlinux-nginx-container \
  -p 8083:80 \
  amazonlinux-nginx:v1
f37bd6255b58d3cfce6f78bda102a0242ddc0121af0100bafab6c650962c8e44
ubuntu@ip-172-31-27-223:~$ curl http://localhost:8083
<!DOCTYPE html>
<html>
<head>
    <title>sample-docker</title>
</head>
<body>
    <h1>Welcome to Amazon Linux and Nginx</h1>
    <p>Custom Docker image created successfully.</p>
</body>
</html>
ubuntu@ip-172-31-27-223:~$
```

Steps performed:

1. Created a project directory for the Dockerfile and HTML file.
2. Created a custom `index.html` page.
3. Created a Dockerfile using `amazonlinux:latest` as the base image.
4. Added maintainer metadata using the `LABEL` instruction.
5. Installed Nginx using the `dnf` package manager.
6. Copied the HTML file into Nginx's default web directory.
7. Exposed container port 8083.
8. Configured Nginx to run in the foreground.
9. Built the image using `docker build`.

10. Started a container and tested the custom web page.

Pushed Docker Image to Public Docker Hub

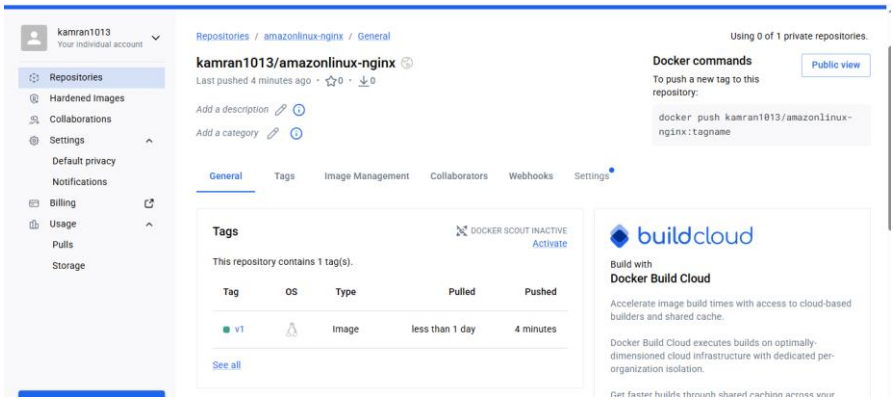
1. Objective

The objective of this task is to upload a custom Docker image to a public Docker Hub repository so that it can be stored, shared, and pulled from other systems.

Created a Public Repository on Docker Hub

1. Open https://hub.docker.com/ in a web browser.
2. Sign in to the Docker Hub account.
3. Navigate to the repositories section.
4. Create a repository named `amazonlinux-nginx`.
5. Select **Public** as the visibility.
6. Create the repository.

```
IMAGE                                ID             DISK USAGE   CONTENT SIZE   EXTRA
amazonlinux-nginx:v1                 62c3f69e0e61       354MB         86.8MB    U
amazonlinux:latest                   8ed3c0a99684       227MB         59.3MB
hello-world:latest                   5e2309035332      25.9kB         9.49kB    U
jenkins/jenkins:lts                  a660310e39ad       720MB          246MB
kamran1013/amazonlinux-nginx:v1      62c3f69e0e61       354MB         86.8MB    U
nginx:latest                         f9ea18bfa4fa       243MB         66.7MB    U
sonarqube:community                  2e378f7852b6      2.56GB          1.1GB
tomcat:9.0                           cd946066142a       641MB          173MB    U
ubuntu:latest                        f144425ff09b       162MB         45.6MB
ubuntu@ip-172-31-27-223:~$ sudo docker push kamran1013/amazonlinux-nginx:v1
The push refers to repository [docker.io/kamran1013/amazonlinux-nginx]
2c281468e883: Pushed
7234d0a9b6ba: Mounted from library/amazonlinux
e19e6bcd0db1: Pushed
v1: digest: sha256:62c3f69e0e61b3f48e39181b3877b6b1c7badc7df7b582e07ed9f7d9fe69889d size: 971
ubuntu@ip-172-31-27-223:~$
```



Push the Image to Docker Hub

Execute: `sudo docker push kamran1013/amazonlinux-nginx`: Docker uploads the image layers to the public Docker Hub repository.

Pushed Docker Image to AWS ECR

Objective: To upload a custom Docker image to AWS Elastic Container Registry (ECR).

Steps:

1. Verified the local Docker image using `sudo docker images`.
2. Attached the IAM role `ecr` to the EC2 instance.
3. Added the required ECR permissions policy to the IAM role.
4. Verified the ECR repository `amazonlinux-nginx`.
5. Authenticated Docker with AWS ECR using AWS CLI.
6. Tagged the Docker image with the ECR repository URL.
7. Pushed the image to ECR using `docker push`.
8. Verified the `v1` image tag in the AWS ECR console.

Result: Successfully pushed the custom Docker image `amazonlinux-nginx:v1` to AWS EC

```
ubuntu@ip-172-31-27-223:~$ aws ecr get-login-password --region us-east-1 \
  | sudo docker login \
      --username AWS \
      --password-stdin 600178453284.dkr.ecr.us-east-1.amazonaws.com

WARNING! Your credentials are stored unencrypted in '/root/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
ubuntu@ip-172-31-27-223:~$ sudo docker push \
  600178453284.dkr.ecr.us-east-1.amazonaws.com/amazonlinux-nginx:v1
The push refers to repository [600178453284.dkr.ecr.us-east-1.amazonaws.com/amazonlinux-nginx]
2c281468e883: Pushed
e19e6bcd0db1: Pushed
7234d0a9b6ba: Pushed
v1: digest: sha256:62c3f69e0e61b3f48e39181b3877b6b1c7badc7df7b582e07ed9f7d9fe69889d size: 971
ubuntu@ip-172-31-27-223:~$
```

